Aman Shaikh

F-02

**AIM: Implement feature techniques(Standardization, Min-Max Scaling ) and observe their effect on model performance**

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix, precision_score, recall_score, f1_score, classificat
```

```
import pandas as pd
data = load_wine()
x=pd.DataFrame(data.data,columns=data.feature_names)
y=data.target
#split
x_train,x_test,y_train,y_test=train_test_split(
    x,y,test_size=0.3,random_state=42
)
```

```
print("\n----- BEFORE SCALING -----")
print(x_train.head())
```

```
----- BEFORE SCALING -----
     alcohol  malic_acid   ash  alcalinity_of_ash  magnesium  total_phenols  \
138    13.49        3.59  2.19               19.5       88.0           1.62
104    12.51        1.73  1.98               20.5       85.0           2.20
78     12.33        0.99  1.95               14.8      136.0           1.90
36     13.28        1.64  2.84               15.5      110.0           2.60
93     12.29        2.83  2.22               18.0       88.0           2.45

     flavanoids  nonflavanoid_phenols  proanthocyanins  color_intensity   hue  \
138        0.48                  0.58             0.88             5.70  0.81
104        1.92                  0.32             1.48             2.94  1.04
78         1.85                  0.35             2.76             3.40  1.06
36         2.68                  0.34             1.36             4.60  1.09
93         2.25                  0.25             1.99             2.15  1.15

     od280/od315_of_diluted_wines  proline
138                          1.82    580.0
104                          3.57    672.0
78                           2.31    750.0
36                           2.78    880.0
93                           3.30    290.0
```

```
#KNN without Scaling
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(x_train, y_train)
acc_no = accuracy_score(y_test,knn.predict(x_test))
print("\nAccuracy WITHOUT scaling:",acc_no)
```

```
Accuracy WITHOUT scaling: 0.7407407407407407
```

```
#Standard Scaling
scaler_std = StandardScaler()
x_train_std = scaler_std.fit_transform(x_train)
```

```
x_test_std = scaler_std.transform(x_test)
x_train_std_df = pd.DataFrame(x_train_std, columns=x_train.columns)
```

```
print("\n----- AFTER  STANDARD SCALING -----")
print(x_train_std_df.head())
```

```
----- AFTER  STANDARD SCALING -----
    alcohol  malic_acid       ash  alcalinity_of_ash  magnesium  \
0  0.628447    1.081206 -0.652127           0.000000  -0.841477
1 -0.540882   -0.612994 -1.427534           0.288180  -1.037487
2 -0.755657   -1.287031 -1.538306          -1.354445   2.294697
3  0.377877   -0.694972  1.747940          -1.152719   0.595936
4 -0.803385    0.388952 -0.541355          -0.432270  -0.841477

   total_phenols  flavanoids  nonflavanoid_phenols  proanthocyanins  \
0      -1.003358   -1.517062              1.711448        -1.230771
1      -0.112585   -0.086751             -0.350476        -0.195036
2      -0.573329   -0.156280             -0.112562         2.014532
3       0.501741    0.668135             -0.191866        -0.402183
4       0.271369    0.241029             -0.905609         0.685339

   color_intensity       hue  od280/od315_of_diluted_wines   proline
0         0.333174 -0.641378                     -1.070901 -0.518219
1        -0.933495  0.346530                      1.330768 -0.215063
2        -0.722384  0.432435                     -0.398434  0.041960
3        -0.171658  0.561293                      0.246586  0.470333
4        -1.296056  0.819008                      0.960225 -1.473819
```

```
from sklearn.preprocessing import MinMaxScaler
#Min-Max Scaling (0-1 range)
scaler_minmax = MinMaxScaler()
x_train_mm = scaler_minmax.fit_transform(x_train)
x_test_mm = scaler_minmax.transform(x_test)
x_train_mm_df= pd.DataFrame(x_train_mm, columns=x_train.columns)
print("\n----- AFTER MINMAX SCALING -----")
print(x_train_mm_df.head())
```

```
----- AFTER MINMAX SCALING -----
    alcohol  malic_acid       ash  alcalinity_of_ash  magnesium  \
0  0.647368    0.549898  0.443850           0.458763   0.195652
1  0.389474    0.171079  0.331551           0.510309   0.163043
2  0.342105    0.020367  0.315508           0.216495   0.717391
3  0.592105    0.152749  0.791444           0.252577   0.434783
4  0.331579    0.395112  0.459893           0.381443   0.195652

   total_phenols  flavanoids  nonflavanoid_phenols  proanthocyanins  \
0       0.220690    0.029536              0.849057         0.145570
1       0.420690    0.333333              0.358491         0.335443
2       0.317241    0.318565              0.415094         0.740506
3       0.558621    0.493671              0.396226         0.297468
4       0.506897    0.402954              0.226415         0.496835

   color_intensity       hue  od280/od315_of_diluted_wines   proline
0         0.437086  0.268293                      0.201465  0.237983
1         0.132450  0.455285                      0.842491  0.310481
2         0.183223  0.471545                      0.380952  0.371946
3         0.315673  0.495935                      0.553114  0.474389
4         0.045254  0.544715                      0.743590  0.009456
```

```
knn.fit(x_train_std, y_train)
acc_std = accuracy_score(y_test, knn.predict(x_test_std))
print("\nAccuracy WITH Standard Scaling:", acc_std)
```

```
Accuracy WITH Standard Scaling: 0.9629629629629629
```

```
scaler_minmax = MinMaxScaler()
x_train_mm = scaler_minmax.fit_transform(x_train)
x_test_mm = scaler_minmax.transform(x_test)
x_train_mm_df = pd.DataFrame(x_train_mm, columns=x_train.columns)
print(x_train_mm_df.head())
```

```
    alcohol  malic_acid       ash  alcalinity_of_ash  magnesium  \
0  0.647368    0.549898  0.443850           0.458763   0.195652
1  0.389474    0.171079  0.331551           0.510309   0.163043
2  0.342105    0.020367  0.315508           0.216495   0.717391
3  0.592105    0.152749  0.791444           0.252577   0.434783
4  0.331579    0.395112  0.459893           0.381443   0.195652

   total_phenols  flavanoids  nonflavanoid_phenols  proanthocyanins  \
0       0.220690    0.029536              0.849057         0.145570
1       0.420690    0.333333              0.358491         0.335443
2       0.317241    0.318565              0.415094         0.740506
3       0.558621    0.493671              0.396226         0.297468
4       0.506897    0.402954              0.226415         0.496835

   color_intensity       hue  od280/od315_of_diluted_wines   proline
0         0.437086  0.268293                      0.201465  0.237983
1         0.132450  0.455285                      0.842491  0.310481
2         0.183223  0.471545                      0.380952  0.371946
3         0.315673  0.495935                      0.553114  0.474389
4         0.045254  0.544715                      0.743590  0.009456
```

```
knn.fit(x_train_mm, y_train)
acc_mm = accuracy_score(y_test, knn.predict(x_test_mm))
print("\nAccuracy WITH MinMaxScaler:", acc_mm)
```

```

Accuracy WITH MinMaxScaler: 0.9444444444444444
```

```
#Confusion Matrix
cm = confusion_matrix(y_test, knn.predict(x_test_mm))
print("Confusion Matrix:")
print(cm)
#Accuracy
acc= accuracy_score(y_test, knn.predict(x_test_mm))
print("\nAccuracy:", acc)
#Precision
prec = precision_score(y_test, knn.predict(x_test_mm), average='macro')
print("\nPrecision:", prec)
#Recall
rec = recall_score(y_test, knn.predict(x_test_mm), average='macro')
print("\nRecall:", rec)
#F1 Score
f1 = f1_score(y_test, knn.predict(x_test_mm), average='macro')
print("\nF1 Score:", f1)
#full classification report
print("\nClassification Report:")
print(classification_report(y_test, knn.predict(x_test_mm)))
```

```
Confusion Matrix:
[[19  0  0]
 [ 1 18  2]
 [ 0  0 14]]

Accuracy: 0.9444444444444444

Precision: 0.9416666666666668

Recall: 0.9523809523809524

F1 Score: 0.9435897435897435
```

```
Classification Report:
              precision    recall  f1-score   support

           0       0.95      1.00      0.97        19
           1       1.00      0.86      0.92        21
           2       0.88      1.00      0.93        14

    accuracy                           0.94        54
   macro avg       0.94      0.95      0.94        54
weighted avg       0.95      0.94      0.94        54
```